

1 Setup & running

Install	python.org 3.10+ (tick Add to PATH) or Anaconda
Libraries	<code>pip install pandas numpy matplotlib openpyxl</code>
Windows	<code>py file.py · py -m pip install ...</code>
macOS/Linux	<code>python3 file.py</code>
Jupyter/Colab	<code>%pip install ...</code> · Shift+Enter runs a cell
Interpreter	<code>python → >>> · exit()</code> to leave
Run scripts from the folder that holds their data files.	

2 Variables & types

<code>company = "Cedar Trading" # str</code> <code>employees = 48 # int</code> <code>revenue = 2_750_000.50 # float (_ = spacer)</code> <code>is_paid = False # bool: True / False</code> <code>VAT_RATE = 0.11 # CONSTANT in capitals</code>			
<code>int("25")</code>	25	<code>float("12.5")</code>	12.5
<code>str(312.5)</code>	'312.5'	<code>round(3.14159, 2)</code>	3.14
<code>type(19.99)</code>	float	<code>bool("")</code>	False
snake_case names · no spaces, no leading digit · case-sensitive.			

3 Operators

<code>+ - * /</code>	add, subtract, multiply, divide (/ gives a float)
<code>//</code>	floor division: <code>150 // 12 → 12</code>
<code>%</code>	remainder: <code>150 % 12 → 6</code>
<code>**</code>	power: <code>1.06 ** 5 → 1.338...</code>
<code>== != < <= > >=</code>	compare → True / False
<code>and or not</code>	combine conditions
<code>in / not in</code>	membership: <code>"late" in text</code>
<code>+= -= *= /=</code>	balance <code>+= 1200</code>
Floats are approximate: <code>0.1 + 0.2 → 0.30000000000000004</code> . For exact money use <code>Decimal("0.10")</code> .	

4 Strings

<code>code = "INV-2026-00457"</code> <code>code[0]</code> # 'I' first character <code>code[-1]</code> # '7' last character <code>code[4:8]</code> # '2026' slice [start:end] <code>code[-5:]</code> # '00457' last five <code>len(code)</code> # 14 <code>"Hi " + name</code> # join <code>"-" * 30</code> # repeat	
<code>.strip()</code>	remove outer spaces
<code>.lower()</code> <code>.upper()</code> <code>.title()</code>	change case
<code>.replace(" ", " ")</code>	replace text
<code>.split(",")</code>	text → list
<code>", ".join(items)</code>	list → text
<code>.startswith("INV")</code>	True / False
<code>.find("@")</code>	position, or -1
<code>.count("a")</code>	occurrences
<code>.isdigit()</code> <code>.isalpha()</code>	content tests
Strings are immutable: store the result, e.g. <code>city = city.upper()</code> .	

5 f-string formatting

<code>print(f"{{client}} owes {{balance:,.2f}} USD")</code>		
CODE	VALUE	SHOWS
<code>{{x:,.2f}}</code>	1845320.456	1,845,320.46
<code>{{x:.1%}}</code>	0.0735	7.3%
<code>{{x:,.0f}}</code>	51750.4	51,750
<code>{{n:06d}}</code>	57	000057
<code>{{n:02d}}</code>	3	03
<code>{{s:<10}}</code>	'North'	left in 10
<code>{{x:>10,.2f}}</code>	186000	right in 10
<code>{{s:^34}}</code>	'TITLE'	centred in 34

6 Input & output

```
name = input("Name: ") # always text
qty = int(input("Qty: ")) # convert!
price = float(input("Price: "))
print("Jan", "Feb", sep=" | ") # Jan | Feb
print("Loading", end="...") # no newline
```

7 Decisions

```
if spend >= 5000:
    tier = "Platinum"
elif spend >= 2500: # first True wins
    tier = "Gold"
else:
    tier = "Bronze"

if 70 <= score < 80: ... # chained
status = "Overdrawn" if bal < 0 else "OK"
if not email: print("Missing") # "" is False

match code: # Python 3.10+
    case "NEW": msg = "Received"
    case "SHP": msg = "Shipped"
    case _: msg = "Unknown"

Falsy: 0 0.0 "" [] {} None False
```

8 Loops

```
for region in regions:
for month in range(1, 13): ... # 1..12
for i, p in enumerate(products, start=1):
for p, price in zip(products, prices):

total = 0 # accumulator
for sale in sales:
    if sale <= 0:
        continue # skip this one
    if sale > 50_000:
        break # stop the loop
    total += sale

while savings < target: # repeat
    savings *= 1.07

range(start, stop, step) excludes stop.
```

9 Lists & tuples

```
sales = [42000, 38500, 45100]
sales.append(51200) # add at end
sales.insert(0, 40000) # add at position
sales.remove(38500) # delete by value
last = sales.pop() # remove last
sales.sort(reverse=True) # in place
sales[0], sales[-1], sales[1:3]
sum(sales), max(sales), min(sales), len(sales)
sorted(sales, reverse=True)[:3] # top 3
copy = sales.copy() # real copy (not =)

vat = [round(p * 1.11, 2) for p in prices]
big = [p for p in prices if p >= 100]

point = (33.89, 35.50) # tuple: fixed
lat, lon = point # unpack
```

10 Dictionaries & sets

```
p = {"sku": "LAP-01", "price": 949.0}
p["price"] = 899.0 # change / add
del p["sku"] # remove
p.get("stock", 0) # default if missing
for key, value in p.items(): ...
p.keys(), p.values()

counts = {} # counting
for item in orders:
    counts[item] = counts.get(item, 0) + 1
best = max(counts, key=counts.get)

eur = {k: v * 0.92 for k, v in usd.items()}

ids = set(purchase_log) # unique values
a & b # in both a | b # in either
a - b # in a, not b
```

11 Functions

```
def line_total(qty, price, discount=0.0):  
    """Return the order-line total."""  
    return round(qty * price * (1 - discount), 2)  
  
line_total(4, 25)           # 100.0  
line_total(4, 25, 0.10)    # 90.0  
line_total(price=25, qty=4) # keywords  
  
def summarize(v):           # 3 results  
    return min(v), max(v), sum(v) / len(v)  
low, high, avg = summarize([120, 340, 95])  
  
def net(rev: float, cost: float) -> float:  
    return rev - cost        # type hints  
  
sorted(staff, key=lambda e: e[1]) # lambda
```

Variables created inside a function are **local**. No **return** → returns **None**.

12 Modules (standard library)

```
import math  
math.ceil(1030 / 48)      # 22: always round up  
math.sqrt(144), math.exp(0.15)
```

```
import statistics as st  
st.mean(x), st.median(x), st.stdev(x)
```

```
import random  
random.seed(42)           # reproducible  
random.randint(1, 100); random.choice(names)  
random.uniform(9, 13); random.sample(ids, 5)
```

```
from datetime import date, datetime  
from datetime import timedelta  
due = date(2026, 8, 15) + timedelta(days=30)  
(date.today() - due).days    # days overdue  
due.strftime("%d %B %Y")     # 14 September 2026  
datetime.strptime("03/15/2026", "%m/%d/%Y")
```

Own module: save `finance_tools.py` in the same folder →
`import finance_tools as ft.`

13 Files

```
with open("notes.txt", "w") as f: # overwrite  
    f.write("Review Q3 budget\n")  
# "a" = append "r" = read (the default)  
with open("notes.txt") as f:  
    for line in f:  
        print(line.strip())  
  
import csv  
with open("sales.csv", newline="") as f:  
    for row in csv.DictReader(f): # row=dict  
        units = int(row["units"]) # text-int  
with open("out.csv", "w", newline="") as f:  
    cols = ["region", "revenue"]  
    w = csv.DictWriter(f, fieldnames=cols)  
    w.writeheader(); w.writerows(rows)
```

```
import json  
with open("config.json", "w") as f:  
    json.dump(config, f, indent=2)  
with open("config.json") as f:  
    config = json.load(f)
```

```
from pathlib import Path  
Path("reports").mkdir(exist_ok=True)  
for p in Path("exports").glob("*.csv"):  
    print(p.name)
```

14 Errors & exceptions

```
try:  
    qty = int(input("Quantity: "))  
except ValueError as e:  
    print("Please enter a whole number.")  
else:  
    print("Accepted")           # only if no error  
finally:  
    print("Done")               # always runs
```

```
if amount > balance:  
    raise ValueError("Insufficient funds.")  
assert 0 <= rate <= 1, "rate must be 0-1"
```

SyntaxError	typo, missing) or quote
IndentationError	use 4 spaces per level
NameError	variable not defined / misspelt
TypeError	wrong type, e.g. "a" + 5
ValueError	right type, bad value: int("ten")
KeyError / IndexError	missing dict key / list position
ZeroDivisionError	dividing by 0
FileNotFoundError	wrong folder or file name
ModuleNotFoundError	pip install the library

Read a traceback from the **bottom line up**.

15 Classes (OOP)

```
class Account:  
    bank = "Cedar Bank" # shared by all  
    def __init__(self, owner, balance=0.0):  
        self.owner = owner # instance data  
        self._balance = balance # _ = internal  
    def deposit(self, amount): # method  
        if amount <= 0:  
            raise ValueError("Must be positive")  
        self._balance += amount  
    @property  
    def balance(self): # read-only value  
        return self._balance  
    def __str__(self): # used by print()  
        return f"{self.owner}: {self._balance:,.}"  
  
class Savings(Account): # inheritance  
    def __init__(self, owner, balance, rate=0.03):  
        super().__init__(owner, balance)  
        self.rate = rate  
  
acct = Savings("Lea", 1000)  
acct.deposit(250); print(acct, acct.balance)
```

16 pandas

```
import pandas as pd  
df = pd.read_csv("sales.csv") # read_excel(*)  
df.shape; df.head(); df.dtypes; df.describe()  
df["product"]                 # one column  
df[["region", "units"]]       # several  
df.loc[2]; df.iloc[0:2]      # label / position  
df["revenue"] = df["units"] * df["unit_price"]  
df[df["revenue"] > 1000]      # filter  
df[(df["region"] == "North") & (df["units"] > 2)]  
df[df["region"].isin(["East", "South"])]  
df.sort_values("revenue", ascending=False)  
df.groupby("region")["revenue"].sum()  
df.groupby("product").agg(  
    orders=("order_id", "count"),  
    revenue=("revenue", "sum"))  
pd.pivot_table(df, values="revenue",  
    index="region", columns="product",  
    aggfunc="sum", fill_value=0)  
df.isna().sum()              # missing values  
df["city"] = df["city"].fillna("Unknown")  
df["name"].str.strip().str.title()  
df["date"] = pd.to_datetime(df["date"])  
df["date"].dt.month_name()  
df.to_csv("out.csv", index=False)  
df.to_excel("out.xlsx")      # (*)
```

(*) Excel needs `pip install openpyxl`. Combine filters with `&` / `|` and brackets, never `and` / `or`.

17 matplotlib

```
import matplotlib.pyplot as plt  
plt.figure(figsize=(8, 4))  
plt.plot(months, revenue, marker="o",  
    label="Actual")  
plt.plot(months, target, linestyle="--",  
    label="Target")  
plt.title("Monthly revenue")  
plt.xlabel("Month"); plt.ylabel("USD '000")  
plt.legend(); plt.grid(alpha=0.3)  
plt.tight_layout()  
plt.savefig("chart.png", dpi=150) # save file  
plt.show()                       # open window
```

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))  
axes[0].plot(x, y); axes[1].bar(labels, vals)  
df.plot(kind="bar", stacked=True) # pandas
```

Trend over time	<code>plt.plot</code>
Compare categories	<code>plt.bar</code> / <code>plt.barh</code>
Relationship	<code>plt.scatter</code>
Distribution	<code>plt.hist</code>
Share of whole (few parts)	<code>plt.pie</code>

18 Business formulas in Python

Price + VAT	<code>net * (1 + VAT_RATE)</code>
Discounted price	<code>price * (1 - rate)</code>
% change	<code>(new - old) / old</code>
Compound interest	<code>p * (1 + r) ** n</code>
Present value	<code>fv / (1 + r) ** n</code>
Loan payment	<code>p * r / (1 - (1 + r) ** -n)</code>
Break-even units	<code>fixed / (price - var_cost)</code>
Gross margin	<code>(price - cost) / price</code>
Markup price	<code>cost * (1 + markup)</code>
Straight-line depr.	<code>(cost - salvage) / life</code>
Boxes needed	<code>math.ceil(units / box_size)</code>
Average	<code>sum(x) / len(x)</code>

19 Debugging checklist

1. Read the **last line** of the error, then its line number.
2. Check colons, brackets, quotes and indentation.
3. `print(type(x), x)` to see what a variable really holds.
4. Did you convert `input()` text to a number?
5. Right folder? Right file name? Library installed?
6. Change one thing, run again, and predict the result first.